



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Lenguajes de Programación
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Peñafiel Solórzano Leyber Smith
Asignatura:	Algoritmos y lógica de programación
Docente:	Ing. José Ruben Caizabuano, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Conocer los lenguajes de programación de la actualidad y sus antecesores.

Específicos:

- Analizar la evolución histórica de los lenguajes de programación.
- Comparar las características principales de distintos lenguajes de programación contemporáneos.
- Aplicar herramientas digitales para sintetizar información en una infografía.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 2

No presenciales: 0

2.4 Instrucciones

- El trabajo se desarrollará de manera individual.
- Realice una consulta sobre LA HISTORIA Y EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN.
- Seleccionar una herramienta que permita realizar infografías.
- Realizar una infografía a manera de resumen de la lectura realizada.
- Subir el documento al aula virtual en el formato de Guías APE en PDF.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Inteligencia artificial, TAC
- Inteligencia artificial, TAC
- Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas (fuentes confiables)

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): Plataforma web y aplicación de diseño gráfico en línea (Canva).

2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas.



2.7 Resultados obtenidos

Conoce los lenguajes de programación de la actualidad y sus antecesores.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

El conocimiento de los lenguajes de programación permite elegir el que mejor se adapte a las necesidades del cliente en proyectos de software.

2.10 Recomendaciones

Sin recomendaciones, todo excelente de la actividad de la APE.

2.11 Referencias bibliográficas

Referencias en formato IEEE

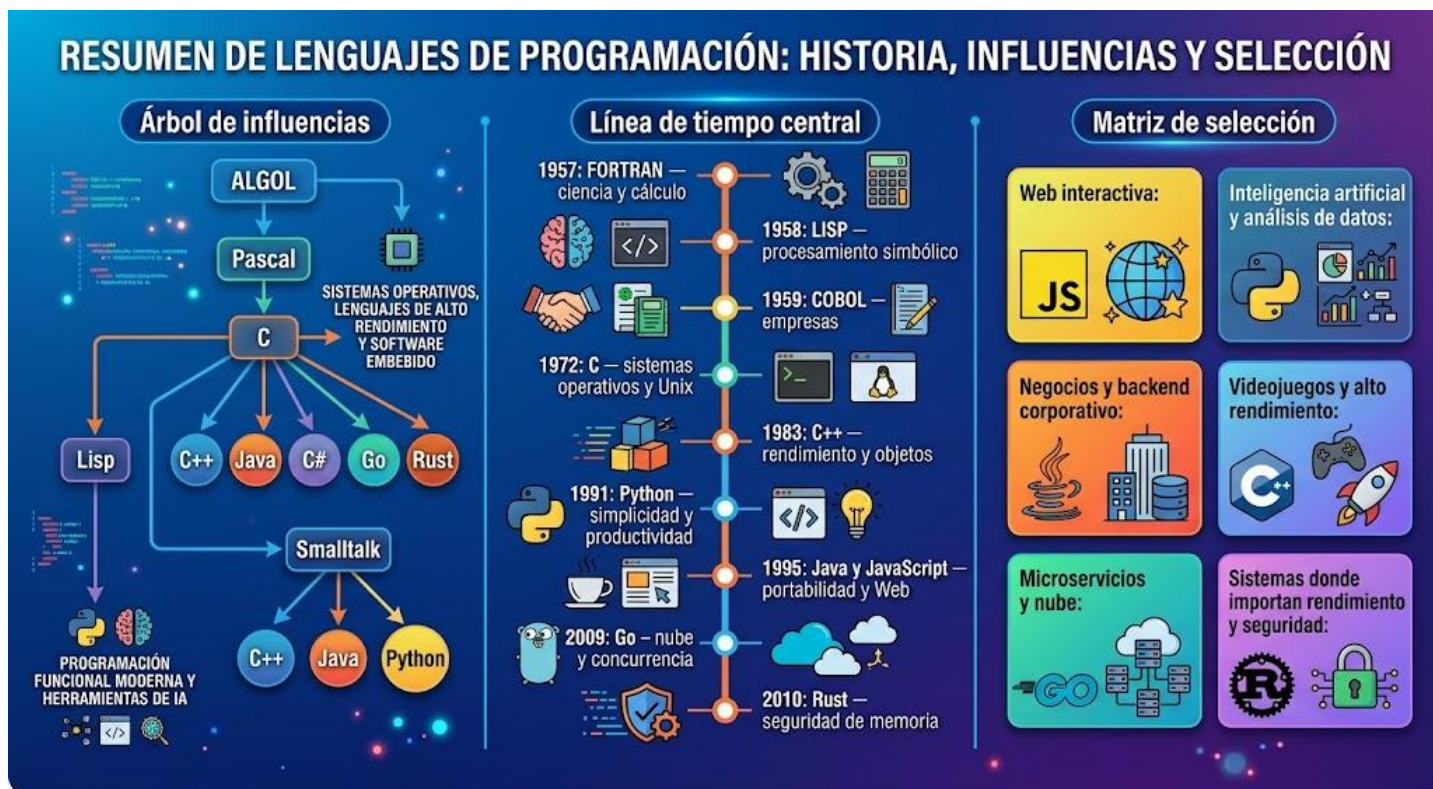
- [1] T. J. Bergin y R. G. Gibson, Eds., *History of programming languages---II*. New York, NY, USA: Association for Computing Machinery, 1996. doi: <https://doi.org/10.1145/234286>.
- [2] J. E. Sammet, «Programming languages: history and future», *Commun. ACM*, vol. 15, n.º 7, pp. 601-610, jul. 1972, doi: 10.1145/361454.361485.
- [3] R. L. Wexelblat, Ed., *History of programming languages*. New York, NY, USA: Association for Computing Machinery, 1978.
- [4] «Report on HOPL IV—ACM SIGPLAN History of Programming Languages Conference». Accedido: 22 de agosto de 2026. [En línea]. Disponible en: <https://www.computer.org/csdl/magazine/an/2021/03/09546088/1x6zF4OWy7S>
- [5] «A History of the History of Programming Languages – Communications of the ACM». Accedido: 22 de agosto de 2026. [En línea]. Disponible en: <https://cacm.acm.org/news/a-history-of-the-history-of-programming-languages/>
- [6] «Evolución del lenguaje de programación: de Fortran a Python, y lo que viene - Tecnología - ABC Color», ABC. Accedido: 22 de agosto de 2026. [En línea]. Disponible en: <https://www.abc.com.py/tecnologia/2026/01/19/evolucion-del-lenguaje-de-programacion-de-fortran-a-python-y-lo-que-viene/>
- [7] «ACM SIGPLAN history of programming languages conference, Los Angeles, California - 102799666 - CHM». Accedido: 22 de agosto de 2026. [En línea]. Disponible en: <https://www.computerhistory.org/collections/catalog/102799666>
- [8] «Why Are There So Many Programming Languages? – Communications of the ACM». Accedido: 22 de agosto de 2026. [En línea]. Disponible en: <https://cacm.acm.org/blogcacm/why-are-there-so-many-programming-languages/>



- [9] «Historia de los lenguajes de programación», *Wikipedia, la enciclopedia libre*. 19 de agosto de 2026. Accedido: 22 de agosto de 2026. [En línea]. Disponible en: https://es.wikipedia.org/w/index.php?title=Historia_de_los_lenguajes_de_programaci%C3%B3n&oldid=174929694
- [10] «The Python Tutorial», Python documentation. Accedido: 22 de agosto de 2026. [En línea]. Disponible en: <https://docs.python.org/3/tutorial/index.html>
- [11] «Java SE 8 Platform Overview». Accedido: 22 de agosto de 2026. [En línea]. Disponible en: <https://docs.oracle.com/javase/8/docs/technotes/guides/index.html>
- [12] «JavaScript | MDN», MDN Web Docs. Accedido: 22 de agosto de 2026. [En línea]. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [13] «Documentation - The Go Programming Language». Accedido: 22 de agosto de 2026. [En línea]. Disponible en: <https://go.dev/doc/>
- [14] «Fearless Concurrency - The Rust Programming Language». Accedido: 22 de agosto de 2026. [En línea]. Disponible en: <https://doc.rust-lang.org/book/ch16-00-concurrency.html>

2.12 Anexos

Infografía





LA HISTORIA Y EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN.

La evolución de los lenguajes de programación muestra un cambio desde instrucciones cercanas al hardware hacia herramientas más legibles, seguras, portables y especializadas. Para cumplir el objetivo general, conviene presentar tanto una línea histórica como una comparación de lenguajes actuales y su relación con sus antecesores.

Evolución histórica

Etapas	Lenguajes representativos	Aporte principal
Décadas de 1940–1950	Lenguaje máquina, ensamblador	Programación directa mediante códigos numéricos o mnemónicos; dependía totalmente del hardware.
Finales de 1950	FORTRAN (1957), LISP (1958), COBOL (1959)	Nacen los lenguajes de alto nivel. FORTRAN se orientó a ciencia e ingeniería; LISP, al procesamiento simbólico; COBOL, a aplicaciones de negocio. FORTRAN incorporó uno de los primeros compiladores optimizadores de alta calidad. [1]
Década de 1960	ALGOL, BASIC, Pascal	Se extendieron la programación estructurada, la enseñanza de programación y la descripción formal de algoritmos. ALGOL influyó en numerosas familias posteriores.[2]
Década de 1970	C (1972), Smalltalk	C surgió en Bell Labs para implementar Unix y facilitó la creación de software de sistemas portable y eficiente. Smalltalk popularizó ideas fundamentales de la programación orientada a objetos. [3]
Décadas de 1980–1990	C++, Java, Python, JavaScript	C++ amplió C con orientación a objetos; Java impulsó la portabilidad mediante la JVM; Python priorizó legibilidad y rapidez de desarrollo; JavaScript llevó la programación dinámica a la Web. [4]
Desde 2000	C#, Go, Rust, Kotlin, TypeScript	Se enfatizan la productividad, la concurrencia, la seguridad de memoria, la computación en la nube y el desarrollo web a gran escala. Rust, por ejemplo, busca prevenir errores de memoria y concurrencia desde la compilación. [5]



Relación entre antecesores y lenguajes actuales

- **FORTRAN** estableció el uso de lenguajes de alto nivel para cálculos científicos; su enfoque sigue vigente en simulación, ingeniería y cómputo numérico.
- **COBOL** fue diseñado para el procesamiento empresarial y administrativo; representa el origen de muchos sistemas de información corporativos y financieros. Sus especificaciones fueron aceptadas por CODASYL en 1959. [6]
- **ALGOL** influyó en la sintaxis estructurada y en lenguajes como Pascal, C, C++ y Java.
- **C** es uno de los antecesores más influyentes: inspiró directamente a C++ y dejó huella en Java, C#, JavaScript, Go y Rust. Fue creado entre 1969 y 1973 junto con Unix.[7]
- **C++** incorporó abstracción y orientación a objetos a la base de C, influyendo en el diseño de lenguajes modernos de alto rendimiento.
- **Java** consolidó el modelo de aplicaciones ejecutadas sobre una máquina virtual y con recolección automática de memoria. [8]
- **Python** y JavaScript representan la tendencia hacia sintaxis accesible, tipado dinámico y desarrollo rápido para datos, automatización y Web.[9]

Comparación de lenguajes contemporáneos

Lenguaje	Tipo de tipado	Paradigmas principales	Ejecución	Fortalezas	Usos frecuentes	Antecedentes/influencias
Python	Dinámico	Imperativo, orientado a objetos, funcional	Interpretado	Sintaxis clara, bibliotecas amplias, desarrollo rápido	IA, ciencia de datos, automatización, Web	ABC, C, Modula-3, Lisp
Java	Estático y fuerte	Orientado a objetos, concurrente	Compila a bytecode para JVM	Portabilidad, ecosistema empresarial, recolección de basura	Backend, sistemas empresariales, Android histórico	C++, Smalltalk
JavaScript	Dinámico	Imperativo, funcional, orientado a objetos basada en prototipos	Interpretado o compilación JIT	Lenguaje principal de interacción web; funciona también en	Frontend, backend, aplicaciones web	Scheme, Self, Java; no es una versión de Java



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



Lenguaje	Tipo de tipado	Paradigmas principales	Ejecución	Fortalezas	Usos frecuentes	Antecedentes/influencias
				servidores con Node.js		
C++	Estático	Procedural, orientado a objetos, genérico	Compilado a código máquina	Alto rendimiento y control de recursos	Videojuegos, motores gráficos, sistemas embebidos, software de alto rendimiento	C
Go	Estático	Imperativo, concurrente	Compilado a código máquina	Compilación rápida, simplicidad, goroutines y canales para concurrencia	Servicios en la nube, redes, microservicios, DevOps	C, Pascal, CSP
Rust	Estático y fuerte	Imperativo, funcional, concurrente	Compilado a código máquina	Seguridad de memoria sin recolector de basura; evita numerosos errores de concurrencia antes de ejecutar	Sistemas, herramientas, navegadores, software crítico	C++, ML, Haskell

Python combina tipado dinámico, interpretación y una sintaxis orientada a la legibilidad, por lo que es útil cuando se busca desarrollar y probar soluciones rápidamente. [10]

Java es un lenguaje de propósito general, orientado a objetos, basado en clases y de tipado fuerte; normalmente se compila a bytecode para ejecutarse sobre la JVM, lo que favorece la independencia de plataforma. [11]

JavaScript funciona con prototipos, es dinámico y admite estilos imperativo, funcional y orientado a objetos. Aunque se asocia con páginas web, también se utiliza fuera del navegador mediante entornos como Node.js. [12]

Go es compilado, de tipado estático y posee mecanismos de concurrencia; incluye recolección automática de memoria. [13]

Rust utiliza un sistema de propiedad y tipos que permite detectar durante la compilación muchos problemas de seguridad de memoria y concurrencia.[14]



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



Link de GitHub

https://github.com/LeyberP/Unidad_1_Primer_Parcial/tree/main/APE/SW-AyLP-APE-01%3ALenguajes_de_Programaci%C3%B3n